

Bharat Calendar AI

System Overview:

Bharat Calendar AI is an intelligent conversational system designed to provide astrological and Hindu calendar information in many Indian languages. The system acts as an interface between users and astrological databases, using artificial intelligence to understand queries and give the responses naturally. The proposed work allows the users to ask questions naturally just like how we ask a friend who has knowledge about this. The AI understands the intent behind the questions, fetches accurate data from sources and presents it in a conversational, easy manner in the user's own language or requested language.

Core Functionality:

The proposed work performs three primary functions each with specific underlying process.

- 1. Query Understanding:** It receives user's question in any natural language. The system doesn't just match keywords; it uses advanced natural language processing to understand intent and context. Here are the steps that are done when user sends query.
 - **Language Detection:** The AI analyses the script and linguistic patterns to identify which of the supported languages the user is asking. This happens automatically without the user specifying their language.
 - **Intent Recognition:** The system determines what the user wants to know. For example, if user prompts "When is Diwali?" then it recognises that user needs festival information.
 - **Entity Extraction:** The system identifies specific details in the query like zodiac signs, dates, time periods, festival names.
 - Traditional systems use simple keyword matching but Bharath Calendar AI uses a Large Language Models (LLM) that understands semantic meaning, handles ambiguity, and maintains conversational context.

2. Data Retrieval: Fetches relevant information from external astrological databases. Once the system understands what the user need, it doesn't directly generate the information. It follows the below steps:

- Tool selection: Based on the identified intent, the agent chooses the appropriate data-fetching function like for Horoscope query it uses get_horoscope tool, same with the other three functions.
- Parameter Preparation: The agent prepares the exact parameters needed for the API call. Formats dates correctly, converts zodiac names to expected format and Maps language codes to API specifications.
- API communication: The tool makes an HTTP request to specialized endpoints.
 - Example: GET <https://api.exaweb.in:3004/api/rashi>
 - Parameters: {
 - "rashi": "LEO",
 - "language": "HI",
 - "day": "26-10-2025"
 - }
- Response validation: The tool checks if the response is valid or not by checking whether the HTTP status code is successful or not, whether the response properly formatted JSON.
- Error Handling: If anything goes wrong (network timeout, invalid response, API error), the tool returns a structured error message instead of just crashing.

3. Intelligent Response: Formats data into human-readable answers in the user's preferred language. The raw data from APIs is typically in JSON format with technical field names. The AI transforms this into natural, conversational text.

- Data Interpretation: The AI analyses the JSON structure and extracts meaningful information.

- **Natural language Detection:** The AI uses the Sarvam AI language model to create fluent, natural-sounding text in the detected language.
- **Formatting and structure:** The AI give appropriate formatting like section headings for clarity, bullet points for lists, proper spacing and line breaks and emphasis on important information.
- **Cultural sensitivity:** The AI adjusts its language and tone based on cultural context. For example, when discussing auspicious timings in Hindi, it uses appropriate respectful language and traditional terms.
- **Conversational flow:** The response is crafted to fit naturally in the ongoing conversation, using appropriate transition of words and maintaining the consistent flow.

Supported Features:

1. Astrological Services

- **Daily Horoscope Predictions for 12 Zodiac Signs:** This isn't just basic sun sign horoscopes. The system provides comprehensive predictions including:
 - **General Forecast:** Overall outlook for the day.
 - **Financial Guidance:** Money matters and career opportunities.
 - **Health Predictions:** Physical and mental well-being
 - **Lucky Elements:** Lucky numbers, colours, and times

The predictions are fetched from professional astrologers' databases and updated daily, ensuring accuracy and relevance.

- **Hindu Calendar (Panchang) Information:** The Panchang is the traditional Hindu almanac, essential for determining auspicious timings for important activities. The system provides Astronomical Data like sunrise and sunset, temporal elements, Auspicious and Inauspicious Times. As this information is crucial for Hindus planning weddings, starting new ventures, or performing religious ceremonies.
- **Comprehensive Holiday Calendars Across Religions:** The system provides holiday information for India's diverse religious communities:

- Hindu Festivals: Diwali, Holi, Navratri, Ganesh Chaturthi, etc.
- Islamic Holidays: Eid al-Fitr, Eid al-Adha, Ramadan dates, etc.
- Christian Celebrations: Christmas, Easter, Good Friday, etc.
- Government Holidays: Republic Day, Independence Day, Gandhi Jayanti, etc.

Each holiday entry includes exact date (accounting for lunar calendar variations), Significance and meaning, Traditional observances.

- **Monthly Festival Schedules:** Users can query festivals for any month, receiving:
 - Chronological list of festivals
 - Cultural and religious significance
 - Regional variations and names

2. Language Support

12 Languages Supported: English and 11 Indian languages (Hindi, Bengali, Gujarati, Tamil, Telugu, Kannada, Malayalam, Marathi, Oriya, Panjabi).

Workflow of Language Support:

- **Automatic Detection:** The system doesn't require users to select a language. It automatically detects the language from the query's script and linguistic patterns.
- **Native Script Support:** Each language is supported in its native script
- **Consistent Responses:** If a user asks in Hindi, the entire response is in Hindi. If they ask in Tamil, the response is in Tamil. The system never mixes languages in a single response.
- **Language Model Optimization:** The Sarvam AI model is specifically trained on Indian languages, ensuring grammatically correct responses, culturally appropriate expressions and proper use of technical terminology in each language.
- **Multi-Turn Language Consistency:** Even in conversations with multiple back-and-forth exchanges, the system maintains the initial language throughout unless the user switches.

Design Patterns:

1. ReAct Pattern (Reasoning + Acting)

The agent follows a cyclic pattern:

- Reason: Analyse user query then determine what information is needed
- Act: Execute appropriate tool to fetch data
- Reason: Interpret retrieved data and format for user
- Act: Return formatted response

This creates intelligent behaviour where the system "thinks" about what to do rather than following rigid rules.

2. Tool-Based Agent Pattern

Instead of the AI trying to answer from general knowledge, it uses specialized "tools" (functions) to fetch accurate, real-time data. This ensures:

- Factual accuracy (no hallucinations)
- Up-to-date information
- Reliable data sources

3. Middleware Pattern

Cross-cutting concerns (CORS, request validation) are handled by middleware layers before reaching core business logic.

4. Streaming Response Pattern

The agent processes requests in a streaming manner, generating multiple intermediate outputs before producing the final response. The system filters these to return only the final answer.

Project components:

1. **LLM connection:** Establish connection to the AI language model that powers natural language understanding and generation. Loads API credentials from environment variables. Configures Connection to Sarvam AI's Language Model as Sarvam AI is an Indian AI company specializing in language models optimized for Indian languages. Unlike generic models (like OpenAI's GPT) that

may struggle with Hindi, Tamil, or Bengali, Sarvam's models are trained extensively on Indian language data.

Temperature affects token selection probability:

- At temperature 0.0 - deterministic
- At temperature 0.2 - slightly varied
- At temperature 1.0 - creative

For Bharath Calendar AI temperature 0.2 I suggested as we need consistency and accuracy. When user asks same question twice, we should get very similar answers. Factual information like horoscopes should be presented consistently. And there should be no creative interpretations of astrological data.

Setting model parameters: Beyond temperature, there are other parameters (not explicitly set, so they use defaults):

- `max_tokens`: Maximum response length (default: depends on model)
- `top_p`: Nucleus sampling parameter (alternative to temperature)
- `presence_penalty`: Penalizes repeating topics
- `frequency_penalty`: Penalizes repeating exact phrases

We use defaults for these because:

- `max_tokens`: No artificial limit on response length
- Others: Not needed for our use case

2. Data Tools: The project module provides 4 specialized functions that act as the data retrieval layer for the Bharat Calendar AI system. These functions serve as the bridge between the AI agent and the API, which hosts comprehensive astrological and calendar databases. Each tool is designed as an independent, self-contained function that can be called by the AI agent to fetch specific types of information based on user queries.

- **Tool-1: get_horoscope**

- This tool retrieves daily horoscope predictions for any of the 12 zodiac signs (Aries, Taurus, Gemini, Cancer, Leo, Virgo, Libra, Scorpio, Sagittarius, Capricorn, Aquarius, Pisces). The horoscope data includes comprehensive predictions covering multiple aspects of life including overall forecast, financial outlook, health predictions, and auspicious elements like lucky numbers and colours.
- The AI agent invokes this tool whenever it detects that a user is asking about horoscope predictions.
- This API used for horoscope is maintained by ExaWeb, a specialized provider of astrological data. The predictions are sourced from professional astrologers and updated daily to ensure accuracy and relevance. The API supports all 12 zodiac signs and provides data in multiple Indian languages plus English.
- It accepts three parameters: the zodiac sign, date that defaults to today if not provided, and an optional language code that defaults to English. The function intelligently handles date parsing through the date util library, which accepts flexible formats like "tomorrow", "26-10-2025", or "October 26, 2025", then converts them to the DD-MM-YYYY format required. The tool returns comprehensive horoscope data including an overall prediction for the day, health predictions, lucky number, and lucky colour - all in the requested language. If errors occur during the API call such as network timeouts or connection failures, the tool catches these exceptions and returns informative error messages.

- **Tool-2: get_date_panchang**

- This tool retrieves comprehensive Hindu calendar (Panchang) data for any specified date. The Panchang is the traditional Hindu book used for determining auspicious times for important activities. It provides detailed astronomical and astrological information including sunrise/sunset times, lunar day (Tithi), lunar mansion (Nakshatra),

auspicious combinations (Yoga), and inauspicious time periods like Rahu Kalam.

- The AI agent invokes this tool when users ask about Daily Panchang, Auspicious timings, Hindu calendar details, Specific date queries.
- The data source is ExaWeb Panchang Daily API. This specialized endpoint provides astronomical calculations and astrological timings that are essential for Hindu religious and cultural activities.
- It accepts an optional date parameter (defaulting to today) and a language parameter, then makes an authenticated API request using a hardcoded API key in the headers. The date formatting for this tool differs from `get_horoscope` - it uses DD/MM/YY format instead of DD-MM-YYYY because that's what this particular API endpoint requires. The returned Panchang data is extremely comprehensive, including astronomical timings like sunrise, sunset, moonrise, and moonset, along with important Hindu calendar elements such as Tithi (lunar day), Nakshatra (lunar mansion), Yoga (auspicious combination), and Karana (half of Tithi). It also provides critical information about inauspicious time periods like Rahu Kalam, Yamagandam, and Gulika Kalam (times to avoid starting important activities), as well as the Abhijit Muhurta (the most auspicious time of the day, approximately 48 minutes around noon). This tool is particularly important because millions of Hindus consult Panchang data before planning weddings, starting businesses, performing ceremonies, or making major life decisions.

- **Tool-3: `get_holidays`**

- This tool retrieves a comprehensive list of holidays for an entire year across multiple religious and cultural categories. It provides information about Hindu festivals, Islamic holidays, Christian celebrations, and government holidays, making it a complete holiday calendar for India.
- The AI agent calls this tool when users ask about Yearly holidays, Specific category, Government holidays and Planning queries.
- The data source is ExaWeb Panchang Holiday API.

- It accepts an optional year parameter that defaults to the current year if not specified, and a language parameter for translating holiday names. The function makes an authenticated API request and returns holidays organized into four main categories: Hindu festivals (like Diwali, Holi, Janmashtami), Islamic holidays (like Eid al-Fitr and Eid al-Adha), Christian celebrations (like Christmas and Easter), and government holidays (like Republic Day and Independence Day). Each holiday entry includes the date in ISO format (YYYY-MM-DD), the holiday name in the requested language, and a brief description of its significance. This makes the tool valuable for year-round planning, allowing users to ask questions like "What are all the holidays in 2025?" and receive a complete, categorized list. The tool typically returns holidays in total year, providing a comprehensive calendar that respects India's religious and cultural diversity.

- **Tool-4: get_monthly_festivals**

- This tool retrieves a detailed list of festivals occurring in a specific month. Unlike get_holidays which provides yearly data across categories, this tool focuses on providing comprehensive festival information for a single month, including regional variations and lesser-known festivals.
- The AI agent invokes this tool for queries like Monthly festival requests, Current month, Specific month-year and Planning queries.
- The data source is ExaWeb Panchang Festival API.
- It accepts three optional parameters: year (defaulting to current year), month (defaulting to current month), and language. The month parameter is particularly smart - it automatically converts the current month to lowercase format using Python function if none is provided, and it normalizes any provided month name to lowercase to ensure API compatibility. The returned festival data for the specified month includes each festival's exact date, name, religious or cultural significance, regional name variations in different languages, and celebration details describing how the festival is typically observed.

This level of detail is particularly useful for users planning around specific months, asking questions like "What festivals are in June?" or "Show me next month's festivals." The tool handles month specification flexibly, accepting queries for the current month, specific named months, or relative references like "next month" that the agent calculates before calling the tool.

All four tools share several important design features that ensure reliability and user-friendliness. First, they implement comprehensive error handling using try-except blocks that catch both specific network-related exceptions (like timeouts and connection failures) and general exceptions, returning descriptive error messages instead of crashing the system. Second, they all return data as JSON strings rather than Python objects, which provides consistency across all tools and makes the data easier for the language model to process. Third, every tool supports multiple languages through language parameters, allowing the same function to return data in English, Hindi, Tamil, Bengali, or any of the other supported languages simply by changing a parameter value. Fourth, they all implement sensible defaults - dates default to "today", years default to the current year, months default to the current month, and languages default to English - which reduces complexity for the agent and creates a more natural user experience where users can be implicit about common assumptions rather than having to specify every detail explicitly.

3. System Prompt: Defines how the agent should behave. This is like a detailed job description and training manual for the AI.

Key Instructions:

- Tool-First Approach: Always use tools to get data, never rely on general knowledge.
- Language Matching: Detect user's language and respond in the same language.
- Data Interpretation: Convert raw JSON into beautiful, readable text.
- Transparency: If data not found, say so clearly.

4. Agent Assembly: Combines LLM, tools and prompt into a functioning agent

It's Process:

- Takes the configured language model.
- Gives it access to all 4 tools.

- Provides the system prompt as instructions.
- Creates a "ReAct agent" that can reason and act.

Output:

- Receive a user message.
- Decide which tool to use (or if any tool is needed).
- Execute the tool.
- Format the response.
- Return the answer.

Complete Dataflow and Processing:

- 1. User Sends Request:** The user sends a JSON-formatted HTTP POST request containing their message and conversation history to the API endpoint. This enables multi-turn conversational interactions.
- 2. FastAPI Receives & Validates:** The FastAPI server receives the request and validates its structure using Pydantic models to ensure all required fields are present and correctly formatted, returning errors if validation fails.
- 3. Context Injection:** The FastAPI server receives the request and validates its structure using Pydantic models to ensure all required fields are present and correctly formatted, returning errors if validation fails.
- 4. Agent Analysis:** The AI agent analyzes the enriched message history to understand the user's intent, identify necessary parameters, and decide which specialized tool is needed to fetch the requested information.
- 5. Agent Response:** If any required information is missing, the agent formulates a clarifying question; otherwise, it proceeds to call the appropriate tool with determined parameters.
- 6. Tool execution:** The selected tool processes the request by formatting parameters, making an authenticated API call to the external ExaWeb service, and handling any communication errors.
- 7. Agen processing:** Upon receiving the tool's response, the agent evaluates the returned data, preparing it for natural language formatting or generating an error message if needed.

- 8. Response generation:** The Sarvam AI language model formats the raw data into a well-structured, culturally appropriate, and human-readable text response in the user's language.
- 9. Filtering:** Only the final AI-generated response is extracted from streaming intermediate events, ensuring that the user receives a clean and coherent answer without internal processing details.
- 10. Response to the user:** The formatted response is returned to the user as a JSON object in the HTTP response, concluding the interaction with accurate and contextually relevant information.

Steps to host the project:

Step 1: Connect to Server

SSH into your DigitalOcean droplet or Linux server using the IP address and credentials. This establishes a secure remote connection where you can execute commands on the server.

ssh root@ip_address

Step 2: Update System Packages

Update the system package manager to ensure all available packages are up-to-date and install security patches. This prevents dependency issues and ensures compatibility.

sudo apt update

sudo apt upgrade -y

Step 3: Install Python and Dependencies

Install Python 3, pip (Python package manager), and git (for version control). These are essential tools needed to run the project.

sudo apt install python3 python3-pip python3-venv git -y

Step 4: Verify Python Version

Check that Python 3.10 or higher is installed on the server. This ensures compatibility with all project dependencies and modern Python features.

python3 --version

Step 5: Clone or Upload Project Files

Either clone the project from Git repository or upload the project files to the server. Navigate to the project directory containing the code files (llm.py, tools.py, prompts.py, agent.py, main.py).

```
cd /root/chatdemo/Chat_Bot(LangGraph)/
```

Step 6: Create Virtual Environment

Create an isolated Python environment for this project to avoid dependency conflicts with other applications. This keeps the project's packages separate from system packages.

```
python3 -m venv venv
```

Step 7: Activate Virtual Environment

Activate the virtual environment so that all pip installs commands install packages into this isolated environment rather than globally.

```
source venv/bin/activate
```

Step 8: Install Python Dependencies

Install all required Python packages from the requirements.txt file into the virtual environment.

```
pip install -r requirements.txt
```

Step 9: Configure Environment Variables

Create a .env file in the project directory and add the Sarvam AI API key. This stores sensitive credentials outside the code and keeps them secure.

```
nano .env
```

Step 10: Verify Installation

Test that all components loaded correctly by importing the modules. This ensures there are no import errors or missing dependencies before running the server.

```
python3 -c "from llm import model; print('Installation successful')"
```

Step 11: Start the Application (Testing Mode)

Run the FastAPI application using Uvicorn in the foreground. This is useful for testing and monitoring logs in real-time during development.

```
uvicorn main:api --host 0.0.0.0 --port 8000
```

Step 12: Start the Application (Production - Screen Session)

For production, use a screen session to keep the application running even after disconnecting from SSH. Create a new screen session, start the server, then detach to leave it running.

```
screen -S api
```

```
uvicorn main:api --host 0.0.0.0 --port 8000
```

Step 13: Enable and Start systemd Service

Enable the service to start automatically on server boot and start it immediately. This ensures the application runs continuously.

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable bharat-calendar-ai
```

```
sudo systemctl start bharat-calendar-ai
```

Step 14: Verify Server is Running

Check if the application is listening on port 8000 and running successfully. This confirms the server started without errors.

```
sudo lsof -i :8000
```

Step 15: Test the API

Open a browser or use curl to test if the API is responding correctly. This verifies the server is accessible and working.

```
curl http://localhost:8000/docs
```

```
curl -X POST http://localhost:8000/invoke \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"messages":[{"role":"user","content":"Hello"}]}'
```